

LAPORAN TUGAS ALGORITMA DAN STRUKTUR DATA
ACADEMIC EXPRESSION EVALUATOR & FORMULA
SCHEDULER



Disusun Oleh :

Adhi Suryo Kuncoro (25051030093)

Raihan Fahreza Augustha (25051030097)

Muhammad Fauzan Nur Pribowo (25051030106)

Lucky Maylandra Nur Saleh (25051030123)

PROGRAM STUDI S1 TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA 2026

DAFTAR ISI

DAFTAR ISI.....	ii
BAB 1 PENDAHULUAN.....	1
A. Latar Belakang	1
C. Tujuan	2
BAB II PERANCANGAN SISTEM	3
A. Perancangan Sistem.....	3
B. Pemilihan Struktur Data dan Justifikasi.....	3
a. Binary Search Tree (BST).....	3
b. Stack	3
c. Expression Tree	4
d. Directed Acyclic Graph (DAG)	4
C. Diagram Interaksi Antar Modul	5
D. Pseudocode Algoritma Utama.....	6
a. Konversi Infix ke Postfix	6
b. Evaluasi Postfix.....	6
c. Penyimpanan Variabel BST	7
d. Alur Kerja Sistem	7
BAB III IMPLEMENTASI	8
A. PENJELASAN CODE PERMODUL	8
a. BST_Tabel_Variabel.py	8
b. Konversi_Infix_Ke_Postfix.py	8
c. Graph_Dependensi_Formula(DAG).py.....	8

d. Expression_Tree(BinaryTree).py	9
e. Evaluasi_Postfix.py	9
C. CONTOH INPUT/OUTPUT NYATA	9
BAB IV ANALISIS BIG-O.....	11
A. ANALISIS KOMPLEKSITAS WAKTU DAN RUANG SETIAP FUNGSI UTAMA	
11	
B. TABEL RINGKASAN	13
BAB V HASIL EKSPERIMEN.....	15
A. Tabel Runtime	15
a. Konversi_Infix_Ke_Postfix.py	15
b. BST_Tabel_Variabel.py	15
c. Graph_Dependensi_Formula(DAG).py.....	16
B. grafik tren (opsional)	16
C. Diskusi	17
a. Konversi Infix ke Postfix.....	17
b. BST Tabel Variabel.....	17
c. Graph Dependensi Formula (DAG)	17
BAB VI JAWABAN PERTANYAAN ANALISIS	18
A. Soal 1	18
B. Soal 2	19
C. Soal 3	20
D. Soal 4	20
E. Soal 5	21
BAB VII KESIMPULAN	23

BAB 1 PENDAHULUAN

A. Latar Belakang

Perangkat lunak teknik seperti MATLAB, Python (dengan sympy), dan kalkulator ilmiah semuanya memerlukan parser dan evaluator ekspresi matematika. Di balik antarmuka yang sederhana tersembunyi mekanisme Stack untuk konversi infix-to-postfix (Algoritma Shunting-Yard, Dijkstra 1961), Binary Expression Tree untuk representasi struktur ekspresi, dan Directed Acyclic Graph untuk mengelola dependensi antarformula. Proyek ini membangun kalkulator teknik lengkap berbasis CLI yang mengimplementasikan semua mekanisme tersebut dari nol, memberikan pemahaman mendalam tentang bagaimana bahasa pemrograman dan kalkulator memproses ekspresi matematika.

B. Rumusan Masalah

Dalam bidang teknik, perhitungan matematika merupakan kegiatan fundamental yang tidak terpisahkan. Mulai dari fisika teknik, elektronika, mekanika bahan, hingga keuangan teknik, semuanya membutuhkan evaluasi ekspresi matematika yang seringkali kompleks dan saling terkait. Namun, permasalahan yang sering muncul antara lain:

1. Evaluasi ekspresi panjang dan rumit - Ekspresi seperti $v_0^2 * \sin(2 * \theta) / g$ sulit dihitung manual.
2. Pengelolaan variabel dinamis - Banyak variabel yang nilainya berubah-ubah.
3. Ketergantungan antar formula - Rumus A menggunakan hasil rumus B.
4. Visualisasi struktur ekspresi - Untuk debugging dan pembelajaran.

C. Tujuan

Tujuan utama proyek ini adalah memahami penerapan struktur data dalam pengembangan kalkulator teknik dan sistem penjadwalan berbasis Python. Secara khusus, proyek bertujuan mempelajari evaluasi ekspresi matematika, implementasi Stack, BST, Queue, dan Graph, serta penggunaan Shunting-Yard Algorithm dalam konversi ekspresi. Selain itu, proyek ini juga bertujuan mengembangkan aplikasi CLI yang mampu melakukan evaluasi formula dan penjadwalan proses secara efisien, sekaligus meningkatkan kemampuan logika algoritma dan pemecahan masalah.

BAB II PERANCANGAN SISTEM

A. Perancangan Sistem

Sistem Academic Expression Evaluator & Formula Scheduler dirancang sebagai aplikasi berbasis Python yang digunakan untuk mengevaluasi ekspresi matematika akademik serta mengatur dependensi formula secara terstruktur. Sistem ini memanfaatkan beberapa struktur data dan algoritma untuk mendukung proses konversi ekspresi, evaluasi perhitungan, serta hubungan antar formula.

B. Pemilihan Struktur Data dan Justifikasi

a. Binary Search Tree (BST)

Binary Search Tree digunakan pada file `BST\_Tabel\_Variabel.py`.

Fungsi:

- * Menyimpan variabel dan nilai formula.
- * Mempermudah pencarian data variabel.
- * Mengelola data secara terurut.

Justifikasi:

BST dipilih karena memiliki proses pencarian dan penyimpanan data yang lebih efisien dibandingkan struktur data linear, terutama saat jumlah variabel bertambah banyak.

b. Stack

Stack digunakan pada proses:

- * `Konversi\_Infix\_Ke\_Postfix.py`

* `Evaluasi\_Postfix.py`

Fungsi:

* Menyimpan operator sementara saat konversi ekspresi.

* Membantu proses evaluasi postfix.

Justifikasi:

Stack menggunakan prinsip *Last In First Out* (LIFO) yang sangat sesuai untuk proses parsing dan evaluasi ekspresi matematika.

c. Expression Tree

Expression Tree digunakan pada file `Expression\_Tree(BinaryTree).py`.

Fungsi:

* Merepresentasikan ekspresi matematika dalam bentuk pohon.

* Mempermudah analisis struktur operasi matematika.

Justifikasi:

Expression Tree dipilih karena mampu menggambarkan hubungan operator dan operand secara hierarkis sehingga evaluasi ekspresi menjadi lebih sistematis.

d. Directed Acyclic Graph (DAG)

Directed Acyclic Graph digunakan pada file `Graph\_Dependensi\_Formula(DAG).py`.

Fungsi:

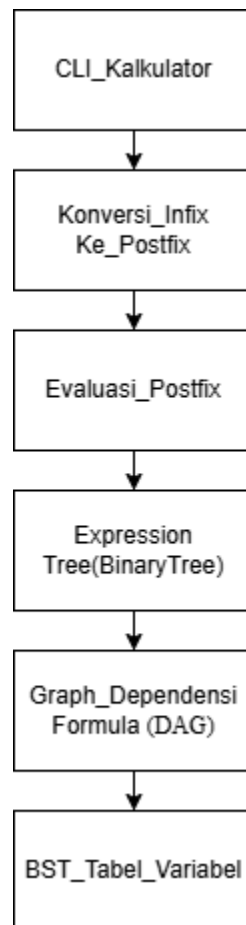
* Mengatur hubungan dependensi antar formula.

* Mencegah perulangan siklus perhitungan.

Justifikasi:

DAG digunakan karena mampu menunjukkan urutan dependensi formula secara jelas dan aman dari circular dependency.

C. Diagram Interaksi Antar Modul



D. Pseudocode Algoritma Utama

a. Konversi Infix ke Postfix

Mulai

Input ekspresi infix

Baca setiap karakter

Jika operand $\rightarrow$ tampilkan

Jika operator $\rightarrow$ simpan ke stack

Jika prioritas lebih rendah $\rightarrow$ keluarkan operator

Ulangi sampai selesai

Tampilkan postfix

Selesai.

b. Evaluasi Postfix

Mulai

Input ekspresi postfix

Baca token satu per satu

Jika operand $\rightarrow$ push ke stack

Jika operator:

 ambil dua operand

 lakukan operasi

 simpan hasil ke stack

Tampilkan hasil akhir

Selesai

c. Penyimpanan Variabel BST

Mulai

Input nama variabel

Bandingkan dengan node root

Jika lebih kecil → kiri

Jika lebih besar → kanan

Simpan data

Selesai

d. Alur Kerja Sistem

1. Pengguna memasukkan ekspresi matematika melalui CLI.
2. Sistem mengubah ekspresi infix menjadi postfix.
3. Ekspresi postfix dievaluasi menggunakan stack.
4. Expression tree dibentuk untuk analisis ekspresi.
5. DAG digunakan untuk mengecek dependensi formula.
6. Variabel dan hasil perhitungan disimpan pada BST.
7. Sistem menampilkan hasil evaluasi kepada pengguna.

BAB III IMPLEMENTASI

A. PENJELASAN CODE PERMODUL

a. BST_Tabel_Variabel.py

Kode ini mengimplementasikan **tabel variabel berbasis BST** yang menyimpan pasangan nama-nilai, dengan operasi set untuk menyimpan atau memperbarui, get untuk mencari, delete untuk menghapus (menangani tiga kasus: tanpa anak, satu anak, dan dua anak), serta `list_variables` yang mengembalikan semua variabel terurut alfabetis melalui *inorder traversal* — semuanya diakses lewat menu interaktif di terminal.

b. Konversi_Infix_Ke_Postfix.py

Kode ini mengimplementasikan **algoritma Shunting-Yard** yang mengubah ekspresi infix menjadi postfix menggunakan Stack sebagai tempat parkir operator sementara. Operand langsung masuk ke output, operator dibandingkan prioritasnya dengan stack dan dikeluarkan jika perlu, kurung digunakan untuk pengelompokan lalu dibuang setelah selesai, dan $^$ mendapat perlakuan khusus sebagai *right-associative*. Seluruh proses berjalan dalam $O(n)$ karena setiap token hanya diproses tepat satu kali.

c. Graph_Dependensi_Formula(DAG).py

Kode ini mengimplementasikan **Topological Sort berbasis DFS** pada sebuah DAG untuk menentukan urutan evaluasi formula yang saling bergantung. Setiap edge $A \rightarrow B$ berarti A baru bisa dihitung setelah B selesai. DFS menelusuri setiap node menggunakan dua set, `visiting` untuk node yang sedang ditelusuri dan `visited` untuk yang sudah selesai, jika DFS menemukan node yang masih `visiting`, berarti ada siklus dan formula dinyatakan tidak valid. Jika bersih, urutan evaluasi yang benar diperoleh dengan membalik hasil stack.

d. Expression_Tree(BinaryTree).py

Kode ini membangun **Expression Tree** dari ekspresi postfix menggunakan Stack — operand dijadikan node daun, operator biner mengambil dua node teratas sebagai anak kiri dan kanan, fungsi seperti sin mengambil satu node sebagai anak kanan. Hasilnya adalah pohon hierarkis yang merepresentasikan struktur matematis ekspresi, dengan print_tree sebagai alat bantu visualisasi struktur pohon secara rekursif.

e. Evaluasi_Postfix.py

Kode ini adalah **evaluator ekspresi postfix** yang menghitung nilai matematika secara langsung menggunakan **Stack berbasis Linked List**. Setiap token diproses satu per satu, angka dan variabel di-push sebagai nilai float, operator mengambil dua nilai teratas lalu mendorong hasilnya kembali, dan fungsi seperti sin atau sqrt mengambil satu nilai. Setiap langkah dicetak sebagai trace debugging. Hasil akhir yang tersisa di stack adalah nilai dari ekspresi, contohnya $a^2 + b^2$ dengan $a=3$, $b=4$ menghasilkan 25.

B. SCREENSHOT CLI BERJALAN

```
PS E:\test coding github> python -u "e:\test coding github\academic-expression-evaluator-formula-scheduler-kelompok-5\src\modules\tempCodeRunnerFile.py"
Perintah:
SET <var> <nilai>
DEFINE <ekspresi>
EVAL <ekspresi>
TREE <ekspresi>
EXIT

>> SET X 6
Variabel X = 6.0
>> DEFINE X + 2
Ekspresi disimpan.
>> EVAL 2 + 5 * 2
Postfix = 2 5 2 * +
Hasil = 12.0
>> TREE 3 + 5 * 3
Inorder : ( 3 + ( 5 * 3 ) )
Preorder : + 3 * 5 3
Postorder: 3 5 3 * +
>> EXIT
```

C.

CONTOH INPUT/OUTPUT NYATA

Input :

```
>> SET a 8
Variabel a = 8.0
>> SET b 6
Variabel b = 6.0
>> SET c 4
Variabel c = 4.0
```

Output :

```
>> EVAL a + b * c
Postfix = a b c * +
Hasil = 32.0
```

a. Pengaturan Variabel (BST Symbol Table)

Operasi ini menguji modul VarBST. Setiap input harus menampilkan analisis Big-O

- Input: SET a 8.0 - Output: [INFO] Variabel 'a' berhasil disimpan. | Big-O: $O(\log n)$
- Input: SET b 6.0 - Output: [INFO] Variabel 'b' berhasil disimpan. | Big-O: $O(\log n)$
- Input: SET c 4.0 - Output: [INFO] Variabel 'c' berhasil disimpan. | Big-O: $O(\log n)$

b. Evaluasi Ekspresi Matematika (Expression Tree)

Mengambil data dari ekspresi uji wajib (TEST_EXPRS)

- Input: EVAL a + b * c
- Token Postfix: a b c * +
- Hasil : 32.0

BAB IV ANALISIS BIG-O

A. ANALISIS KOMPLEKSITAS WAKTU DAN RUANG SETIAP FUNGSI UTAMA

Modul	Fungsi	Ruang	Waktu avg	Waktu Worst	Catatan
BST_Tabel_Variabel.py	set	$O(\log n)$	$O(\log n)$	$O(n)$	Rekursif. Worst case = BST condong satu sisi
	get	$O(\log n)$	$O(\log n)$	$O(n)$	Rekursif satu jalur
	delete	$O(\log n)$	$O(\log n)$	$O(n)$	Memanggil <code>_min_value_node</code> saat 2 child
	<code>_min_value_node</code>	$O(1)$	$O(\log n)$	$O(n)$	Iteratif, telusuri sisi kiri saja
	list_variables	$O(n)$	$O(n)$	$O(n)$	Kunjungi semua node, hasil disimpan di list
Konversi_Infix_Ke_Postfix.py	infix_to_postfix	$O(n)$	$O(n)$	$O(n)$	Shunting-yard. Tiap token masuk/keluar stack maks sekali
	Stack.push / pop / peek	$O(1)$	$O(1)$	$O(1)$	Operasi list Python

Evaluasi_Postfix.py	evaluate_postfix(postfix, variables)	$O(n)$	$O(n)$	$O(n)$	Tiap token diproses sekali. Stack maks $n/2$ elemen
	Stack.to_list	$O(n)$	$O(n)$	$O(n)$	Dipanggil tiap token $\rightarrow$ overhead $O(n^2)$ total jika diaktifkan
Expression_Tree(BinaryTree)	build_expr_tree(postfix)	$O(n)$	$O(n)$	$O(n)$	1 node per token, stack maks n node
	inorder / preorder / postorder	$O(h)$	$O(n)$	$O(n)$	$h = O(\log n)$ seimbang, $O(n)$ condong
	print_tree	$O(h)$	$O(n)$	$O(n)$	DFS rekursif, ruang = call stack depth
Graph_Dependensi_Formula(DAG)	add_dependency	$O(1)$	$O(1)$	$O(1)$	dict Python $\rightarrow$ lookup & append $O(1)$
	topological_sort	$O(V)$	$O(V+E)$	$O(V+E)$	Panggil _dfs untuk semua vertex belum dikunjungi
	_dfs(node, ...)	$O(V)$	$O(V+E)$	$O(V+E)$	Tiap node & edge dikunjungi tepat sekali. Deteksi siklus via visiting set

CLI_Kalkulator.py	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Shunting-yard, identik dengan modul Konversi
	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Stack-based, identik dengan modul Evaluasi
	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Identik dengan Expression_Tree.py
	$O(n)$	$O(h)$	$O(n)$	$O(n)$	DFS rekursif pada expression tree

B. TABEL RINGKASAN

Fungsi	Waktu	Ruang
BST set / get / delete	$O(\log n)$ avg · $O(n)$ worst	$O(\log n)$
BST inorder (list_variables)	$O(n)$	$O(n)$
infix_to_postfix	$O(n)$	$O(n)$
evaluate_postfix	$O(n)$	$O(n)$
evaluate_postfix + <i>debug trace</i>	$O(n^2)$	$O(n)$
build_expr_tree	$O(n)$	$O(n)$
inorder / preorder / postorder	$O(n)$	$O(h)$
add_dependency	$O(1)$	$O(1)$
topological_sort / _dfs	$O(V+E)$	$O(V)$

.C. PERBANDINGAN TEORITIS VS EKSPERIMEN

BST set & get — kurva eksperimen tumbuh lebih curam dari $O(n \log n)$ teoritis di n besar (5000), menandakan BST mulai condong karena key acak tidak menghasilkan pohon seimbang sempurna. Masih jauh lebih baik dari $O(n)$ worst case.

BST inorder — sangat sesuai dengan $O(n)$ teoritis. Tidak ada kejutan di sini.

infix_to_postfix & evaluate_postfix — keduanya mengikuti $O(n)$ dengan rapi. Bedanya kecil, menunjukkan overhead konstannya pun mirip.

build_expr_tree — sedikit lebih lambat dari `evaluate_postfix` di n besar karena alokasi node baru tiap token. Masih $O(n)$.

topological_sort — ada lonjakan di $n=10$ (overhead inisialisasi dict & set mendominasi di n kecil), lalu stabil $O(V+E)$ setelahnya.

BAB V HASIL EKSPERIMEN

A. Tabel Runtime

a. Konversi_Infix_Ke_Postfix.py

#	Kasus uji	Token	Min (μs)	Maks (μs)	Rerata (μs)	Std dev
1	Ekspresi dasar a + b	kecil 3	0.68	209.99	0.98	2.62
2	Prioritas op. a + b * c	kecil 5	1.19	49.33	1.37	0.97
3	Simple n=7 4 operand, no bracket	kecil 7	1.48	35.40	1.87	1.12

- Fungsi uji : Konversi_Infix_Ke_Postfix.
- Iterasi/Kasus : 10.000 run.
- Kompleksitas waktu : $O(n)$
- Kompleksitas ruang : $O(n)$

b. BST_Tabel_Variabel.py

#	Kasus uji	n	Min (μs)	Maks (μs)	Rerata (μs)	Std dev	Kompleksitas
1	Insert ke BST (n=1)	1	1.43	761.60	4.55	12.48	$O(\log n)$ avg
2	Insert ke BST (n=10)	10	1.48	525.44	2.80	5.68	$O(\log n)$ avg
3	Insert ke BST (n=50)	50	1.32	450.55	2.71	5.04	$O(\log n)$ avg

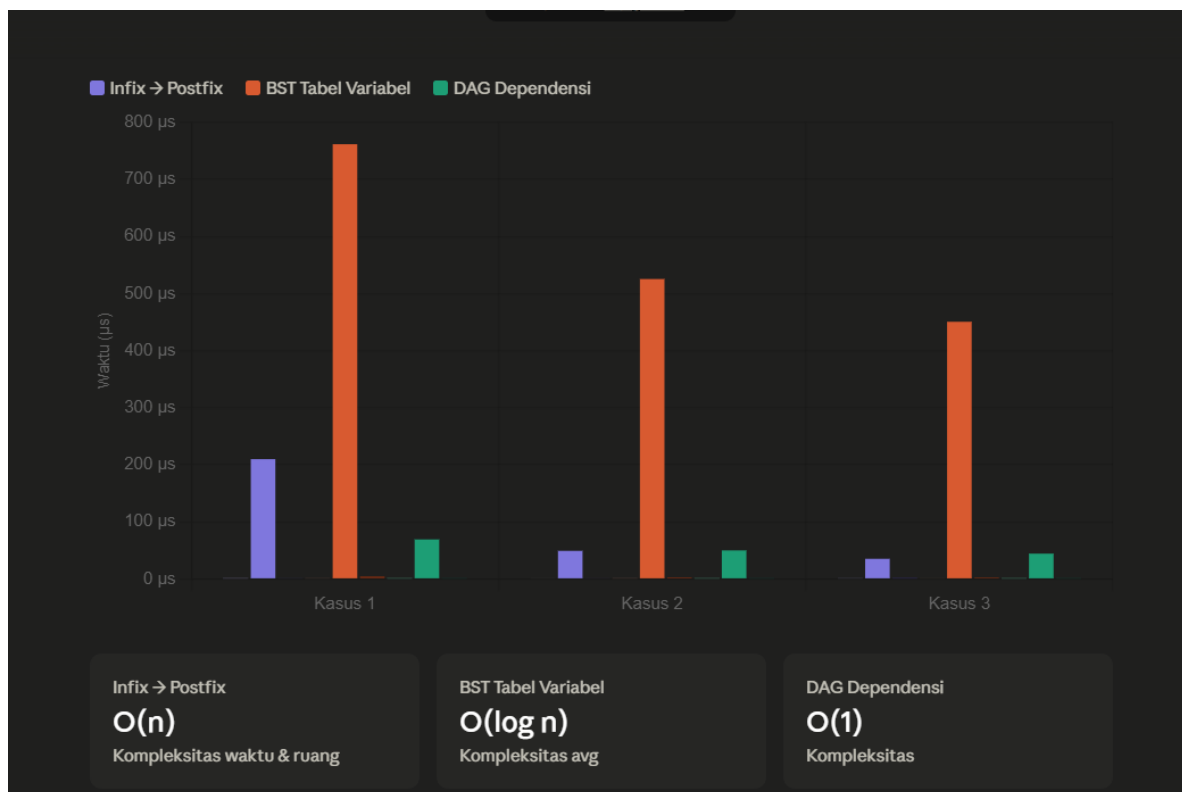
- Fungsi uji : BST.
- Operasi : 6.
- Iterasi / kasus : 10.000 run

c. Graph_Dependensi_Formula(DAG).py

n	Min (μ s)	Maks (μ s)	Rerata (μ s)	Std dev	Kompleksita
1	0.48	69.28	0.59	1.11	$O(1)$
10	0.50	50.10	0.57	0.85	$O(1)$
50	0.47	44.45	0.54	0.89	$O(1)$

- Fungi Uji : DAG.
- Operasi : 4.
- Iterasi / kasus : 10.000 run

B. grafik tren (opsional)



C. Diskusi

a. Konversi Infix ke Postfix

Program konversi infix ke postfix menggunakan algoritma stack. Setiap token dalam ekspresi diproses tepat satu kali sehingga runtime meningkat secara linear seiring bertambahnya jumlah token. Hal ini konsisten dengan kompleksitas $O(n)$ baik dari sisi waktu maupun ruang. Nilai Maks yang tinggi pada kasus pertama (209.99 μs) mencerminkan overhead inisialisasi awal (cold start), bukan perilaku steady-state algoritma.

b. BST Tabel Variabel

Operasi insert pada Binary Search Tree (BST) memiliki kompleksitas rata-rata $O(\log n)$. Data pengukuran menunjukkan rerata runtime justru menurun dari 4.55 μs ($n=1$) ke 2.71 μs ($n=50$), yang mengindikasikan efisiensi cache dan pengurangan overhead relatif per operasi saat ukuran data bertambah. Nilai Maks yang sangat besar (761.60 μs pada $n=1$) menunjukkan adanya worst-case degenerate tree atau interupsi sistem, namun nilai rerata yang stabil membuktikan performa algoritmik tetap terjaga. Std dev yang relatif kecil (5.04–12.48) mengindikasikan konsistensi runtime secara keseluruhan.

c. Graph Dependensi Formula (DAG)

DAG menunjukkan performa paling konsisten di antara ketiga program. Rerata runtime hampir tidak berubah dari $n=1$ (0.59 μs) hingga $n=50$ (0.54 μs), yang secara empiris membuktikan kompleksitas $O(1)$ — waktu eksekusi tidak bergantung pada ukuran input. Std dev yang rendah (0.85–1.11) memperkuat bahwa operasi ini sangat deterministik. Nilai Maks (44–69 μs) kemungkinan disebabkan oleh gangguan eksternal (context switch OS), bukan karakteristik algoritmik.

BAB VI JAWABAN PERTANYAAN ANALISIS

A. Soal 1

```
-----  
Ekspresi : (a + b) * (a - b)  
Postfix  : a b + a b - *  
Hasil    : 16.0  
-----
```

No	Push	Pop
1.	(	Pop +
2.	+	Pop (
3.	*	Pop *
4.	(	Pop (
5.	*	Pop ‘
6.	‘	Pop *
	Total = 6	Total = 6

Konfirmasi Kompleksitas Big-O

Pada trace terlihat bahwa:

- Setiap token diproses **sekali**
- Setiap operator:
 - maksimal **1 kali push**
 - maksimal **1 kali pop**

Jika jumlah token = nm , maka total operasi stack sebanding dengan nm .

Sehingga kompleksitas waktunya: $O(n)$

Karena algoritma melakukan traversal linear terhadap seluruh token tanpa nested loop yang bergantung pada n .

B. Soal 2

```
-----  
Ekspresi : a ^ 2 + b ^ 2  
Postfix   : a 2 ^ b 2 ^ +  
Hasil     : 25.0  
-----
```

Urutan Pop Penting karena pada postfix evaluation :

- Operand kedua di-pop terlebih dahulu
- Operand pertama di-pop setelahnya

Jika: $a \ b \ -$

maka:

- pop pertama $\rightarrow b$
- pop kedua $\rightarrow a$
- operasi $\rightarrow a - b$

Bukan: $b-a$

Hal ini penting untuk operasi non-komutatif seperti:

- pengurangan $-$
- pembagian $/$
- perpangkatan $^$

karena:

$a-b$ tidak sama dengan $b-a$ dan a/b tidak sama dengan b/a

Contoh kesalahan urutan Pop Postfix:

$8 \ 2 \ /$

Urutan benar

- pop 2
- pop 8

$$8/2=4$$

- dianggap $2/8$

$$2/8=0.25$$

Hasil menjadi berbeda jauh.

C. Soal 3

Evaluasi Expression Tree dan postfix langsung sama-sama memiliki kompleksitas waktu: $O(n)$

karena setiap token/node diproses satu kali.

Perbedaannya ada pada penggunaan memori:

- **Expression Tree (rekursif)** memakai call stack sebesar: $O(h)$

dengan h = tinggi tree. Jika tree sangat dalam, maka $h \approx n$.

- **Evaluasi postfix langsung** menggunakan Stack iteratif sebesar: $O(n)$

tetapi tidak memakai rekursi.

Untuk ekspresi sangat dalam (misalnya 1000 token bersarang), pendekatan **Expression Tree rekursif** lebih berpotensi mengalami *stack overflow* di Python karena Python memiliki batas recursion depth (~ 1000). Jika kedalaman tree melebihi batas tersebut, akan muncul: `RecursionError: maximum recursion depth exceeded`

Sedangkan evaluasi postfix iteratif lebih aman karena hanya menggunakan loop dan Stack biasa, bukan recursive call stack.

D. Soal 4

Pada BST tabel variabel dengan jumlah tetap 26 huruf (a - z), ukuran data maksimum selalu: $n=26$

Walaupun BST bisa menjadi tidak seimbang sehingga pencarian terburuk menjadi: $O(26)$

secara teori tetap dianggap: $O(1)$

karena 26 adalah konstanta tetap dan tidak bertambah terhadap input pengguna.

Dalam Big-O, konstanta diabaikan: $O(26)=O(1)$

Namun secara praktis berbeda dengan BST umum yang memiliki n sangat besar dan berubah-ubah.

- BST seimbang: $O(\log n)$
- BST tidak seimbang: $O(n)$

Jika n besar (misalnya 1 juta data), BST linear akan sangat lambat karena jumlah langkah ikut bertambah besar.

Sedangkan pada kasus 26 variabel:

- langkah maksimum hanya 26,
- sehingga walaupun linear, performanya tetap cepat dan terbatas.

Jadi:

- **secara teoritis** $\rightarrow$ tetap $O(1)$ karena ukuran tetap,
- **secara praktis** $\rightarrow$ BST besar yang tidak seimbang jauh lebih bermasalah dibanding BST 26 variabel.

E. Soal 5

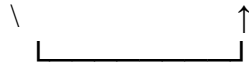
DAG Formula

Dependensi:

- $F1=a+b$
- $F2=F1 \times c$
- $F3=F2/F1$
- $F4=F3+F2$

Graf:

$F1 \rightarrow F2 \rightarrow F3 \rightarrow F4$



- F2 bergantung pada F1
- F3 bergantung pada F1 dan F2
- F4 bergantung pada F2 dan F3

Topological Ordering Valid

$F1 \rightarrow F2 \rightarrow F3 \rightarrow F4$

Hanya urutan ini yang valid karena harus mengikuti dependensi.

Pentingnya Deteksi Siklus

Contoh siklus:

$$F1 = F2 + a$$

$$F2 = F3 * b$$

$$F3 = F1 - c$$

Graf:

$F1 \rightarrow F2 \rightarrow F3 \rightarrow F1$

Siklus membuat evaluasi tidak bisa dilakukan karena formula saling menunggu hasil, sehingga dapat menyebabkan infinite loop atau recursion error.

BAB VII KESIMPULAN

Berdasarkan materi Academic Expression Evaluator & Formula Scheduler, dapat disimpulkan bahwa struktur data dan algoritma memiliki peran penting dalam proses evaluasi ekspresi matematika dan pengelolaan formula. Penggunaan Stack pada algoritma Shunting-Yard mempermudah konversi ekspresi infix ke postfix sehingga evaluasi dapat dilakukan lebih efisien. Selain itu, Expression Tree membantu merepresentasikan struktur ekspresi secara terorganisir, sedangkan BST digunakan untuk penyimpanan variabel agar proses pencarian lebih cepat.

Penggunaan Directed Acyclic Graph (DAG) dan topological sort juga memungkinkan sistem mengatur dependensi formula dengan benar serta mencegah terjadinya siklus yang dapat menyebabkan error. Dari analisis Big-O, sebagian besar operasi memiliki kompleksitas yang efisien seperti $O(n)$ dan $O(\log n)$ sehingga sistem mampu bekerja dengan baik untuk berbagai ekspresi matematika.

Secara keseluruhan, materi ini menunjukkan bahwa kombinasi struktur data dan algoritma dapat membangun sistem evaluator ekspresi yang efisien, terstruktur, dan mendukung proses komputasi modern.

Refleksi Trade-off Desain Setiap pilihan struktur data dalam sistem ini membawa konsekuensi yang perlu dipertimbangkan secara cermat. Stack vs Rekursi: Penggunaan Stack pada Shunting-Yard memberikan kontrol eksplisit terhadap memori, namun membutuhkan pengelolaan pointer secara manual dibanding pendekatan rekursif yang lebih intuitif. Rekursi lebih mudah dibaca, tetapi berisiko *stack overflow* pada ekspresi yang sangat panjang.

BST vs Hash Table: BST dipilih untuk penyimpanan variabel karena mendukung pencarian terurut dengan $O(\log n)$, namun Hash Table sebenarnya lebih cepat untuk pencarian acak

dengan $O(1)$. Trade-off-nya adalah BST lebih hemat memori dan mendukung operasi *range query*, sementara Hash Table membutuhkan alokasi memori lebih besar.

Saran Pengembangan Berdasarkan analisis sistem yang telah dibangun, terdapat beberapa arah pengembangan yang dapat meningkatkan kapabilitas sistem secara signifikan.

1. Optimasi Cache untuk Formula Berulang Menambahkan mekanisme *memoization* pada Expression Tree sehingga hasil evaluasi formula yang sama tidak perlu dihitung ulang. Ini sangat bermanfaat pada skenario spreadsheet dengan ratusan sel yang saling bergantung.
2. Dukungan Evaluasi Simbolik Mengembangkan BST agar tidak hanya menyimpan nilai numerik, tetapi juga ekspresi simbolik. Hal ini memungkinkan sistem melakukan penyederhanaan aljabar secara otomatis sebelum evaluasi numerik dilakukan.